

Intro slides

Computing Gradients

```
In [5]: lin = nn.Linear(10, 1)
        y = lin(torch.randn(1, 10))
        y.backward()
        gradient = lin.weight.grad
```

```
In [6]: x = torch.randn(10)
        y = x.sum()
        print(y)
```

```
tensor(-6.1808)
```

```
In [7]: x = torch.randn(10, requires_grad=True)
        y = x.sum()
        print(y)
```

```
tensor(-2.3190, grad_fn=<SumBackward0>)
```

Controlling the Gradients (no_grad)

```
In [8]: x = torch.randn(10, requires_grad=True)
        # use as context manager
        with torch.no_grad():
            y = x.sum()
        print(y)
```

```
tensor(-3.3742)
```

```
In [9]: # use as decorator
        @torch.no_grad()
        def func(x: torch.Tensor) -> torch.Tensor:
            return x.sum()

        print(func(x))
```

```
tensor(-3.3742)
```

Computing Gradients

```
In [5]: lin = nn.Linear(10, 1)
        y = lin(torch.randn(1, 10))
        y.backward()
        gradient = lin.weight.grad
```

```
In [6]: x = torch.randn(10)
        y = x.sum()
        print(y)
```

tensor(-6.1808)

```
In [7]: x = torch.randn(10, requires_grad=True)
        y = x.sum()
        print(y)
```

tensor(-2.3190, grad_fn=<SumBackward0>)

Controlling the Gradients (detach)

```
In [10]: def func(x: torch.Tensor) -> torch.Tensor:
        y = x.sum()
        return y.detach()

        print(func(x))
```

tensor(-3.3742)

```
In [11]: def func(x: torch.Tensor) -> float:
        y = x.sum()
        return y.item()

        print(func(x))
```

-3.3742122650146484

Controlling the Gradients (zero_grad)

```
In [12]: x = torch.randn(4, requires_grad=True)
y1, y2 = x.sum(), torch.sum(x ** 2)
y1.backward()
print("y1      gradient:", x.grad) # gradient of y1
y2.backward()
print("y1 + y2 gradient:", x.grad) # gradient of y1 + y2

y1      gradient: tensor([1., 1., 1., 1.])
y1 + y2 gradient: tensor([1.3260, 0.2682, 1.1554, 3.0507])
```

```
In [13]: # compute gradient from y1 again (wrong)
y1.backward()
print("wrong  y1 gradient:", x.grad) # gradient of y1 + y2 + y1
# compute gradient from y1 again (right)
del x.grad # reset gradient
y1.backward()
print("correct y1 gradient:", x.grad) # gradient of y1

wrong  y1 gradient: tensor([2.3260, 1.2682, 2.1554, 4.0507])
correct y1 gradient: tensor([1., 1., 1., 1.])
```

Assignment 1: pytorch intro

```
In [4]: class FCNetwork(nn.Module):

    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 500)
        self.fc2 = nn.Linear(500, 300)
        self.fc3 = nn.Linear(300, out_features)
        self.phi = nn.Tanh()
```

```
In [4]: def forward(self, x):
        s1 = self.fc1(x)
        a1 = self.phi(s1)
        s2 = self.fc2(a1)
        a2 = self.phi(s2)
        logits = self.fc3(a2)
        return logits
```

```
In [2]: class FCNetwork(nn.Module):

    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        # do not try this at home!
        self.w1 = nn.Parameter(torch.randn(500, in_features))
        self.b1 = nn.Parameter(torch.randn(500))
        self.w2 = nn.Parameter(torch.randn(300, 500))
        self.b2 = nn.Parameter(torch.randn(300))
        self.w3 = nn.Parameter(torch.randn(out_features, 300))
        self.b3 = nn.Parameter(torch.randn(out_features))
```

```
In [2]: def forward(self, x):
        s1 = torch.addmm(self.b1, x, self.w1.t())
        a1 = torch.tanh(s1)
        s2 = torch.addmm(self.b2, a1, self.w2.t())
        a2 = torch.tanh(s2)
        logits = torch.addmm(self.b3, a2, self.w3.t())
        return logits
```

```
In [3]: class FCNetwork(nn.Module):

    ...

    def probabilities(self, x):
        """ Get the class probabilities from the network. """
        logits = self.forward(x)
        return torch.softmax(logits, dim=-1)

    @torch.no_grad()
    def predict(self, x):
        """ Get a prediction from the network. """
        logits = self.forward(x)
        return torch.argmax(logits, dim=-1)
```

```
In [5]: conv_net = nn.Sequential(
    nn.Conv2d(1, 5, 5),
    nn.AvgPool2d(4),
    nn.ELU(),
    nn.Conv2d(5, 10, 4),
    nn.ELU(),
    nn.Flatten(),
    nn.Linear(90, 10),
    # nn.LogSoftmax()
)
```

```
In [6]: mean, std = .1307, .3081
normalise = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((mean, ), (std, ))
])
```

```
In [7]: dataset = datasets.MNIST("~/pytorch", train=True)
raw_data = dataset.data.float() / 255.
raw_data.mean(), raw_data.std()
```

```
Out[7]: (tensor(0.1307), tensor(0.3081))
```

```
In [ ]: DataLoader(
    dataset,
    batch_size=1,
    shuffle=False,
    num_workers=0,
)
```

```
In [9]: @torch.no_grad()
def evaluate(network: nn.Module, data: DataLoader, metric: callable) -> list:
    with torch.no_grad():
        results = _forward(network, data, metric)
        return [res.item() for res in results]
```

```
In [9]: @torch.no_grad()
def evaluate(network: nn.Module, data: DataLoader, metric: callable) -> list:
    with torch.no_grad():
        results = _forward(network, data, metric)
        return [res.item() for res in results]
```

```
In [9]: def _forward(network: nn.Module, data: DataLoader, metric: callable) -> torch
        """ Helper function to avoid code duplication. """
        for x, y in data:
            logits = network(x)
            res = metric(logits, y)
            yield res
```

```
In [9]: @torch.enable_grad()
def update(network: nn.Module, data: DataLoader, loss: nn.Module,
           opt: optim.Optimizer) -> list:
    errs = []
    for err in _forward(network, data, loss):
        errs.append(err.item())

        opt.zero_grad()
        err.backward()
        opt.step()

    return errs
```

```
In [ ]: ce = nn.CrossEntropyLoss() # if model ends with `nn.Linear`
nll = nn.NLLLoss() # if model ends with `nn.LogSoftmax`
raise NotImplementedError # if model ends with `nn.Softmax`
```

```
In [ ]: ? torch.optim.SGD
```

```
In [ ]: avg_errs = []
errs = evaluate(net, loader, ce, sgd)
avg_errs.append(sum(errs) / len(errs))
for _ in range(num_epochs):
    errs = update(net, loader, ce, sgd)
    avg_errs.append(sum(errs) / len(errs))
```

Assignment 2 Vision Net

```
In [ ]: def _forward(network: nn.Module, data: DataLoader, metric: callable):
        device = next(network.parameters()).device

        for x, y in data:
            x, y = x.to(device), y.to(device)
            # same as before

        @torch.no_grad()
        def evaluate(network: nn.Module, data: DataLoader, metric: callable) -> list:
            network.eval()

            # same as before

        @torch.enable_grad()
        def update(network: nn.Module, data: DataLoader, loss: nn.Module,
                    opt: optim.Optimizer) -> list:
            network.train()

            # same as before
```

```
In [2]: # example of failing code (pytorch bug)
fc = torch.nn.Linear(3, 5)
opt = torch.optim.Adagrad(fc.parameters())

fc.cuda()
fc(torch.randn(1, 3).cuda()).sum().backward()
opt.step()
```

RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cuda:0 and cpu!

```
In [3]: x = torch.randn(1, 1).cuda()
print("detach:", x.detach())
print("item: ", x.item())

detach: tensor([[0.6303]], device='cuda:0')
item: 0.6302676796913147
```

```
In [ ]: gap = nn.AdaptiveAvgPool2d((1, 1))
classifier = nn.Sequential(
    nn.Linear(512, 512),
    nn.ReLU(True),
    nn.Dropout(),
    nn.Linear(512, 512),
    nn.ReLU(True),
    nn.Dropout(),
    nn.Linear(512, num_classes)
)
```

Using pre-trained weights (exercise 3)

```
In [4]: weights = vgg.VGG11_Weights.DEFAULT.get_state_dict(progress=True)
        list(weights.keys())[4] + ["..."] + list(weights.keys())[-4:]
```

```
Out[4]: ['features.0.weight',
         'features.0.bias',
         'features.3.weight',
         'features.3.bias',
         '...',
         'classifier.3.weight',
         'classifier.3.bias',
         'classifier.6.weight',
         'classifier.6.bias']
```

```
In [ ]: def vgg_init_(network: CifarVGG):
        from torchvision.models import vgg
        weights = vgg.VGG11_Weights.DEFAULT.get_state_dict(progress=True)
        selected_weights = ...
        network.features.load_state_dict(selected_weights)

        for m in network.classifier.modules():
            if isinstance(m, nn.Linear):
                nn.init.kaiming_uniform_(m.weight)
                nn.init.zeros_(m.bias)
```

Transfer Learning (exercise 4)

```
In [ ]: normalise = torchvision.transforms.Compose([
        torchvision.transforms.Resize((224, 224)),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize((0.4914, 0.4822, 0.4465), (0.247,
        ])
```

```
In [ ]: c10_train, c10_valid, _ = torch.utils.data.random_split(cifar10, [500,
```

```
In [ ]: model.features.requires_grad_(False)
        # OR
        optimizer(model.classifier.parameters())
```

Skip Connections (Exercise 5)

```
In [ ]: class ResBlock(nn.Module):
        """ Residual block using skip-connections on activation level. """

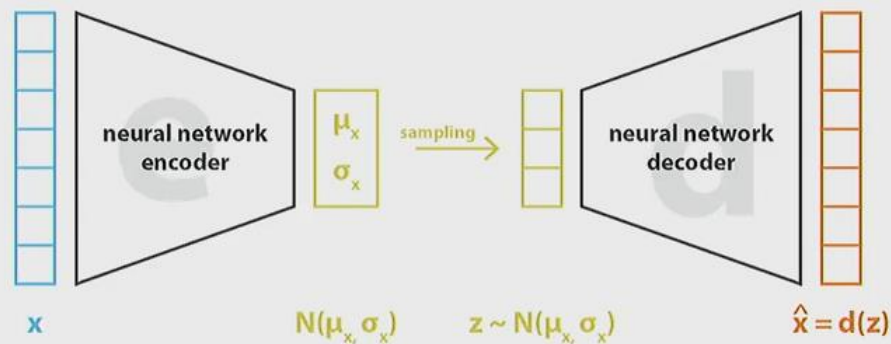
        def __init__(self, in_channels: int, out_channels: int, kernel_size: int,
                        stride: int = 1, phi: nn.Module = nn.ReLU(), extra_params: dict = {}):
            super().__init__()
            conv_kwargs = {"kernel_size": kernel_size, "padding": (kernel_size - 1) // 2}
            self.phi = phi
            self.residual = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, stride=stride, **conv_kwargs),
                phi, nn.Conv2d(out_channels, out_channels, **conv_kwargs),
            )

            self.correction = nn.Identity()
            if in_channels != out_channels or stride > 1:
                self.correction = nn.Conv2d(in_channels, out_channels, 1, stride=stride)

        def forward(self, x):
            return self.phi(self.correction(x) + self.residual(x))
```

Assignment 3: Auto-Encoders

Variational Auto-Encoder



$$\text{loss} = \|x - \hat{x}\|^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = \|x - d(z)\|^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

Transposed Convolution (exercise 1 & 2)

```
In [ ]: def forward(self, x):  
    s = conv(x, self.conv.weight)  
    out, idx = max_pool(s, return_indices=True)  
    a = relu(out)  
    return a, [x.shape[-2:], (idx, s.shape[-2:]), None]
```

```
In [ ]: def undo(self, a, context):  
    x_shape, (idx, s_shape), _ = context  
    out = torch.relu(a)  
    s = max_unpool2d(out, idx, output_size=s_shape)  
    x = conv_transpose2d(s, self.conv.weight, output_padding=...)  
    return x
```

Feature Visualisation (exercise 2)

```
In [ ]: seq = UndoableSequential(net.features)
vis_layer1 = seq.visualise_activations(x_im, layer=1)
vis_layer2 = seq.visualise_activations(x_im, layer=4)
```



Convolutional Auto-Encoder (exercise 3)

```
In [ ]: class AutoEncoder:
    def __init__(self, in_channels, hid_channels, code_channels, ...)
        self.encoder = nn.Sequential(
            nn.Conv2d(in_channels, hid_channels, kernel_size),
            ...
        )

        self.decoder = nn.Sequential(
            ...
            nn.ConvTranspose2d(hid_channels, in_channels, kernel_size)
        )
```

```
In [ ]: def forward(self, x):
    return self.decoder(self.encoder(x))
```

Note on Tied Weights Auto-Encoder

```
In [ ]: class AutoEncoder(nn.Module):
        """ Fully connected auto-encoder with tied weights. """

        def __init__(self, in_features: int, code_features: int,
                        activation: nn.Module = nn.Sigmoid()):
            super().__init__()
            self.encoder = nn.Sequential(
                nn.Linear(in_features, code_features)
            )

            self.activation = activation
            # self.decoder = nn.Sequential(
            #     activation,
            #     nn.Linear(code_features, in_features),
            # )

        def decoder(self, h):
            from torch.nn.functional import linear
            b = self.encoder[0].bias
            w = self.encoder[0].weight
            return linear(self.activation(h) - b, w.t())
```

Auto-Encoder Loss (exercises 4)

```
In [ ]: # option 1 (typical)
auto_encoder.assumption = "gauss"
trainer.criterion = nn.MSELoss()
data.target_transform = data.transform
```

Auto-Encoder Loss (exercises 4)

```
In [ ]: # option 2 (untypical)
auto_encoder.assumption = "binary"
trainer.criterion = nn.BCEWithLogitsLoss()
data.target_transform = transforms.ToTensor()
```

Auto-Encoder Loss (exercises 4)

```
In [ ]: # option 3 (also common)
auto_encoder.assumption = "bernoulli"
trainer.criterion = nn.BCEWithLogitsLoss()
data.target_transform = transforms.ToTensor()
```

Variational Auto-Encoder (exercise 5)

```
In [ ]: def forward(ae, x):  
        code = ae.encoder(x)  
        return ae.decoder(code)
```

```
In [ ]: def forward(vae, x):  
        code_mean, code_log_var = vae.encoder(x)  
        code = vae.sample(code_mean, code_log_var)  
        return vae.decoder(code)
```

Reparametrisation Trick (exercise 5)

```
In [ ]: def sample(vae, mean, log_var):  
        # not differentiable  
        return torch.normal(mean, torch.exp(log_var / 2.))
```

```
In [ ]: def sample(vae, mean, log_var):  
        # differentiable  
        std = torch.exp(log_var / 2.)  
        return mean + std * torch.normal(0, 1, size=mean.shape)
```

Regularisation (exercise 5)

$$\begin{aligned} D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2) \parallel Z) &= \mathbb{E}_{p_{\mathcal{N}}}[\log p_{\mathcal{N}}(X)] - \mathbb{E}_{p_{\mathcal{N}}}[\log p_Z(X)] \\ &= \mathbb{E}_{p_{\mathcal{N}}} \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(X - \mu)^2}{2\sigma^2} \right] - \mathbb{E}_{p_{\mathcal{N}}} \left[-\frac{1}{2} \log(2\pi) - \frac{X^2}{2} \right] \\ &= -\frac{1}{2} \log(2\pi\sigma^2) - \mathbb{E}_{p_{\mathcal{N}}} \left[\frac{X^2 - 2X\mu + \mu^2}{2\sigma^2} \right] + \frac{1}{2} \log(2\pi) + \mathbb{E}_{p_{\mathcal{N}}} \left[\frac{X^2}{2} \right] \\ &= -\frac{1}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \mathbb{E}_{p_{\mathcal{N}}}[X^2] + \frac{\mu}{\sigma^2} \mathbb{E}_{p_{\mathcal{N}}}[X] - \frac{\mu^2}{2\sigma^2} + \frac{1}{2} (\sigma^2 + \mu^2) \\ &= \frac{1}{2} (\mu^2 + \sigma^2 - \log(\sigma^2) - 1) \end{aligned}$$

Assignment 5 : GANs (A4 was optional)

Adversarial Attack (Exercise 1)

```
In [ ]: # enable input gradients directly
x.requires_grad_(True)

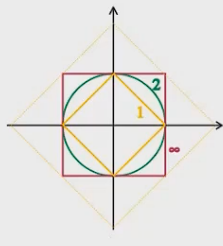
# or alternatively
delta = torch.zeros_like(x).requires_grad_(True)
x_adv = x + delta
```

```
In [ ]: # gradient ascent
with torch.no_grad():
    delta += lr * delta.grad.sign()
```

```
In [ ]: # or gradient hacking (untested)
delta.grad.neg_().sign_()
sgd.step()
```

Adversarial Attack (Exercise 1)

```
In [ ]: # standard projection
with torch.no_grad():
    delta *= epsilon / torch.linalg.norm(delta)
```



```
In [ ]: # projection with infinity norm
with torch.no_grad():
    delta.clamp_(-epsilon, epsilon)
```

Generative Adversarial Examples (Exercise 2)

```
In [ ]: def update_fake_examples(self, network: nn.Module, loss_func: nn.Module,
                                epsilon: float = .1):
    x = self.fake_data.to(device)
    y = torch.zeros(len(x), 1).to(device)

    adv = adversarial_attack(x, y, network, loss_func, epsilon=epsilon)
    self.fake_data = adv.clamp_(x.min(), x.max()).cpu()
```

Generative Adversarial Training (Exercise 3)

```
In [ ]: # lower learning rate -> slow down discriminator
        opt = optim.Adam(discriminator.parameters(), lr=1e-4)
```

```
In [ ]: # higher epsilon -> allow more variety in adversarial examples
        train_discriminator(discriminator, d_loader, bce, opt,
                           num_epochs=15, vis_every=5, epsilon=.7)
```

Generator Error (Exercise 4)

The objective:

$$\min_g \{ \mathbb{E}[\log(1 - d(g(Z)))] \}$$

Remember:

$$\text{BCE}(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

$$\delta^{(L)} = \frac{\partial \text{BCE}}{\partial \mathbf{s}^{(L)}} = \hat{y} - y$$

Discriminator Error (Exercise 4)

The objective:

$$\max_d \mathbb{E}[\log d(X)] + \mathbb{E}[\log(1 - d(g(Z)))].$$

```
In [ ]: fake_x = gan.generator.generate(batch_size=len(real_x), bounded=False)
        fake_logits = gan.discriminator(fake_x)
        real_logits = gan.discriminator(real_x)
```

```
In [ ]: d_err = bce(fake_logits, 0) + bce(real_logits, 1)
```

The game (Exercise 4)

```
In [ ]: for i, (x, _) in enumerate(loader, 1):
        d_err = ...

        opt.zero_grad()
        d_err.backward()
        assert all(not par.grad for par in gan.generator.parameters())
        opt.step()

        g_err = ...

        opt.zero_grad()
        g_err.backward()
        assert all(not par.grad for par in gan.discriminator.parameters())
        opt.step()
```

The game (Exercise 4)

```
In [ ]: for i, (x, _) in enumerate(loader, 1):
        d_err = ...

        opt.zero_grad()
        d_err.backward()
        assert all(not par.grad for par in gan.generator.parameters())
        opt.step()

        g_err = ...

        opt.zero_grad()
        g_err.backward()
        assert all(not par.grad for par in gan.discriminator.parameters())
        opt.step()
```

Assignment 6: Normalizing Flows

First order autograd (exercise 1)

```
In [ ]: def jacobian(func: nn.Module, x: torch.Tensor):
        y = func(x.requires_grad_(True))

        g = torch.zeros_like(y)
        j_rows = []
        for i in range(y.size(-1)):
            g[:, i] = 1.
            grad, = torch.autograd.grad(y, x, g, retain_graph=True)
            j_rows.append(grad)
            g[:, i] = 0.

        return torch.stack(j_rows, dim=-2)
```

Change of Variables (exercise 2)

$$p_Y(\mathbf{y}) = p_X(g^{-1}(\mathbf{y})) \cdot |\det \mathcal{J}_g(\mathbf{y})|^{-1}.$$

```
In [ ]: @torch.no_grad()
        def cov_pdf(y: torch.Tensor, func: nn.Module,
                    x_dist: distributions.Distribution):
            x = func.invert(y)
            x_pdf = x_dist.log_prob(x).sum(-1).exp()
            abs_det = func.log_det_jacobi(x).exp()
            return x_pdf / abs_det
```

Additive Coupling (exercise 3)

```
In [ ]: x1, x2 = torch.chunk(x, 2, dim=1)
        shift = self.conditioner(x2)
        y1, y2 = x1 + shift, x2
        y = torch.cat([y2, y1], dim=1)
```

$$\mathcal{J} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} \mathbf{I} + \mathbf{0} & \mathcal{J}_{\text{cond}} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}$$

Log-likelihood (exercise 4)

- Univariate Gaussian

$$p_{\mathcal{N}}(x; \mu, \sigma^2) = C \exp\left(-\frac{1}{2}(x - \mu)\frac{1}{\sigma^2}(x - \mu)\right)$$

- Multivariate Gaussian

$$p_{\mathcal{N}}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = C \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

```
In [ ]: log_density = self.prior.log_prob(y).sum(-1)
        nll = -log_density - log_det
```

Training the Flow (exercise 5)

```
In [ ]: flow = NormalisingFlow(
        AdditiveCoupling(MultiLayerConditioner(392, 392, depth=3)),
        AdditiveCoupling(MultiLayerConditioner(392, 392, depth=3)),
        AdditiveCoupling(MultiLayerConditioner(392, 392, depth=3)),
        AdditiveCoupling(MultiLayerConditioner(392, 392, depth=3))
    )
```

Note on Identity Jacobian

$$\log p_X(\mathbf{x}; \mathbf{w}) = \log p_Y(g(\mathbf{x}; \mathbf{w})) + \log |\det \mathcal{J}_g(\mathbf{x}; \mathbf{w})|.$$

```
In [ ]: class Scaling(nn.Module):

        def __init__(self, n_in: int):
            super().__init__()
            self.log_scale = nn.Parameter(torch.zeros(1, n_in))

        def _forward(self, x):
            return self.log_scale.exp() * x
```

Images with scaling:

